COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010: Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	9 / 13 / 26	**DATE SUBMITTED:**	9 / 15 / 26
NAME:	Santos, Paterno Miguel R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 8.1: Sorting Algorithms Pt2

1. PROCEDURE OUTPUT

What procedures did you perform?

Code + Console Screenshot

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

int main(){
    const int size = 100;

    int arr[size], shell[size], merge[size], quick[size];
    std::srand(std::time(0));

    for (int i = 0; i < size; i++){
        arr[i] = std::rand() % 100 + 1;
        shell[i] = arr[i];
        merge[i] = arr[i];
        quick[i] = arr[i];
    }

    std::cout << "Array for sort testing: \n\n";
    for (int i = 0; i < size; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
```

```
Array for sort testing:

37 39 24 93 16 9 24 89 32 92 78 62 61 71 50 35 42 24 10 69 89 24 53 47 72 4 10 85 62 56 68 95 66 6 91 88 48 2 39 31 6 24
 27 31 69 3 29 90 4 89 60 59 3 55 26 96 24 35 95 85 90 62 1 47 3 79 37 55 68 41 75 90 30 48 82 81 63 26 70 61 17 43 97 7
3 82 48 3 87 49 100 13 91 69 53 57 73 75 9 68 2
```

Observations

Using arr[size] as a way to insert the same random value array to three separate arrays for different sorting methods and then using ctime for giving out random seeds.

Table 8-1. Array of Values for Sort Algorithm Testing

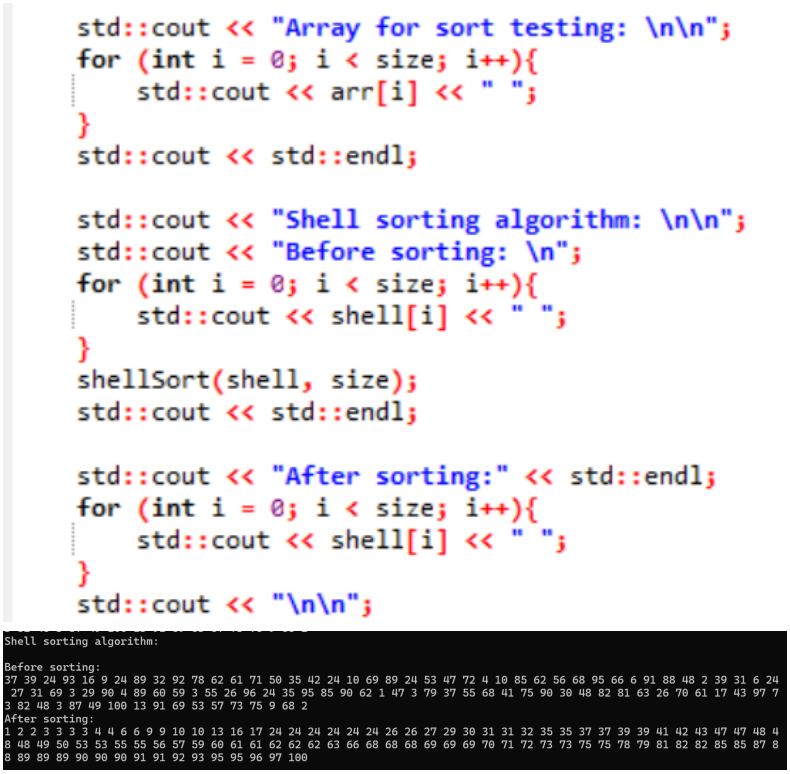
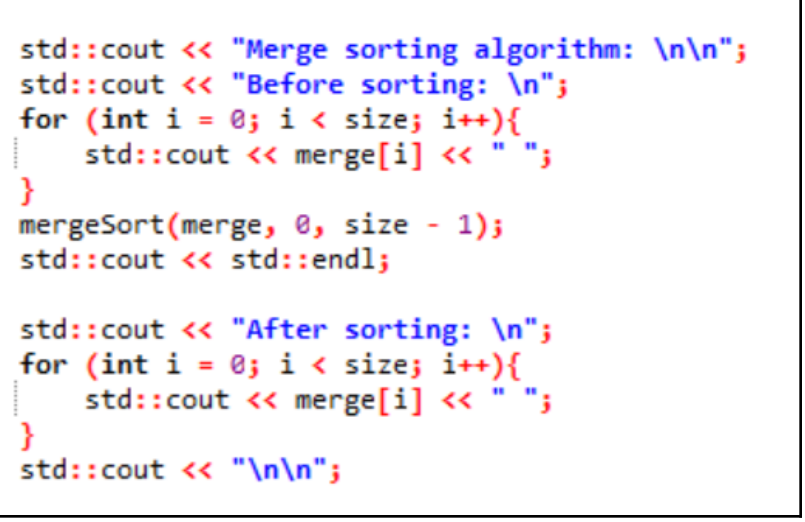
Code + Console Screenshot	
Observations	After using the shellSort function, it uses a gap value of two to swap to values repeatedly until it organizes the entire array.

Table 8-2. Shell Sort Technique

Code + Console Screenshot	

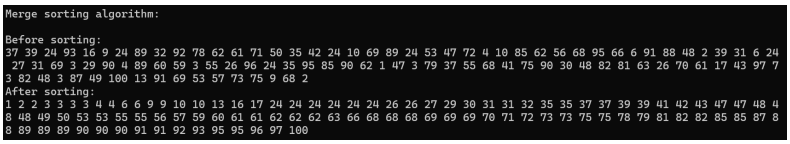
	
Observations	After the mergeSort function is called, it breaks down the array until it only contains one element. Afterwards, it sorts itself with the neighboring group then slowly merges back together as they sort.

Table 8-3. Merge Sort Algorithm

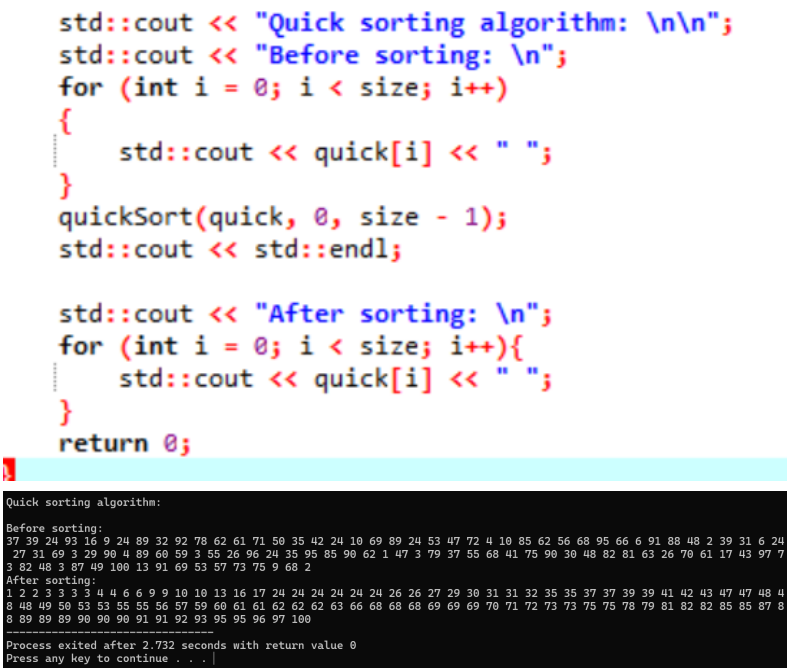
Code + Console Screenshot	
Observations	As the quickSort function is called it divides by selecting one element as a pivot. Then the array is partitioned around the pivot and the values smaller than the pivot go to the left while larger than the pivot go to the right. The pivot is then in its correct position. This process repeats until it fixes the array in ascending order.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Problem 1

```
int main(){
    int arr1[10] = {34, 12, 45, 7, 23, 56, 18, 9, 41, 27};
    std::cout << "Original array: \n";
    dispArray(arr1, 10);

    int pivot = partition(arr1, 0, 9);

    std::cout << "After partition: \n";
    dispArray(arr1, 10);

    std::cout << "Pivot position is " << pivot << std::endl;
    insertionSort(arr1, 0, pivot - 1);
    insertionSort(arr1, pivot + 1, 9);

    std::cout << "Sorting sub lists: \n";
    dispArray(arr1, 10);
    std::cout << std::endl;
```

```
Original array:
34 12 45 7 23 56 18 9 41 27
After partition:
12 7 23 18 9 27 34 45 41 56
Pivot position is 5
Sorting sub lists:
7 9 12 18 23 27 34 41 45 56
```

Problem 2

```
int arr2[15] = {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74};

std::cout << "Original array: \n";
dispArray(arr2, 15);
quickSort(arr2, 0, 14);

std::cout << "Quick Sorted: \n";
dispArray(arr2, 15);
return 0;
```

```
Original array:
4 34 29 48 53 87 12 30 44 25 93 67 43 19 74
Quick Sorted:
4 12 19 25 29 30 34 43 44 48 53 67 74 87 93

--------------------------------
Process exited after 1.2 seconds with return value 0
Press any key to continue . . .
```

Quick sort would have the fastest time performance if a good pivot is selected. Merge and quick sort can have an O(n log n) time complexity since they repeatedly divide the array into smaller parts. The division creates log n while each of them goes around n elements.

Conclusion:

I learned how to use shell, merge and quick sorting works. I understand how the divide and conquering works in quick sorting and how merge sorting works by breaking the array down. But I still need to improve my understanding of recursion and partitions since I am still getting confused about it.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.